



## Provider

Desenvolvimento Móvel

Aula 12/36

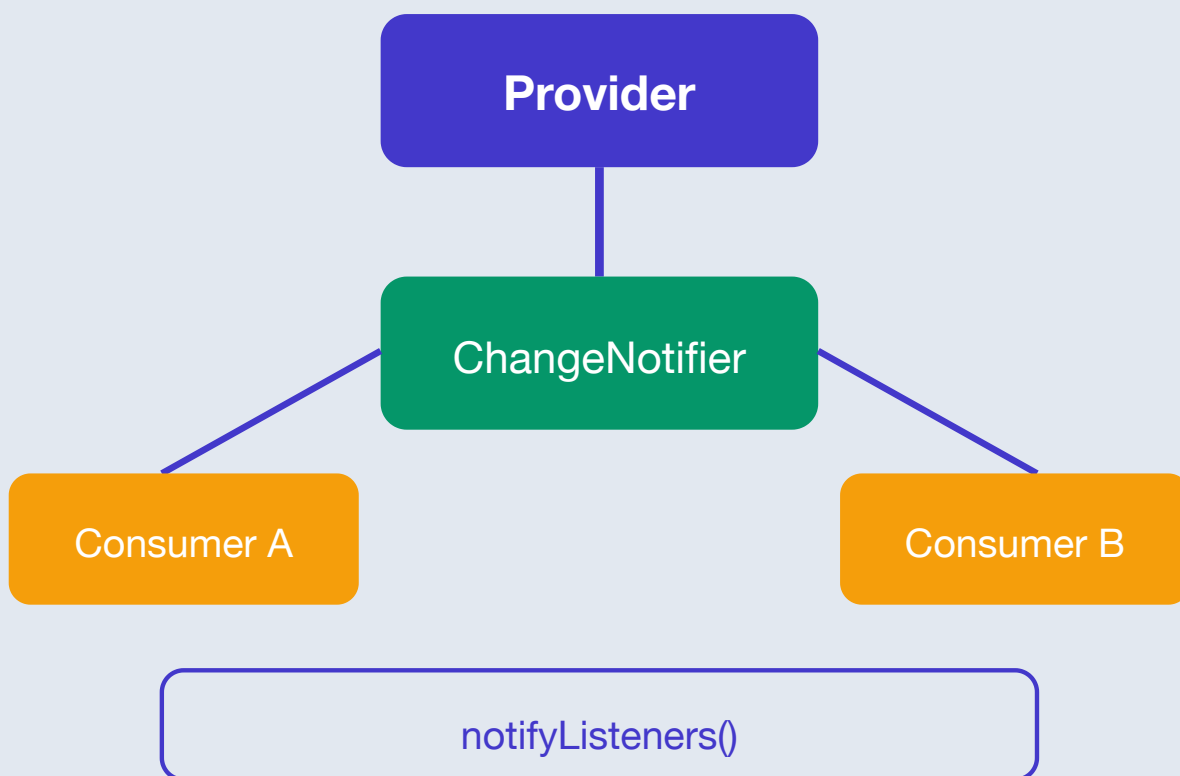
Widget

Provider

ios

### Conceito

Provider é como uma estação de rádio: o `ChangeNotifier` transmite mudanças (`notifyListeners()`) e os widgets sintonizados (Consumer) atualizam automaticamente. `ChangeNotifierProvider` disponibiliza uma instância na árvore de widgets (build context). `Consumer<T>` reconstrói apenas o widget certo quando o estado muda — sem atualizar a tela inteira. `MultiProvider` empilha vários providers como canais diferentes. `context.read<T>()` acessa o provider sem escutar mudanças — útil para ações como chamar métodos. `context.watch<T>()` escuta e reconstrói o widget se o estado mudar — como ficar ouvindo a rádio. Provider é uma camada fina sobre `InheritedWidget`, simplificando algo que antes era complexo.





### Atividade Prática

Migre o cronometro da aula anterior para usar Provider: crie um `TimerModel` extends `ChangeNotifier` com metodos `iniciar()`, `pausar()`, `resetar()`, um `Timer` e a variavel `Duration`. No `MaterialApp`, use `ChangeNotifierProvider<TimerModel>`. No cronometro, use `context.watch<TimerModel>()` para exibir o tempo e `context.read<TimerModel>()` para chamar acoes dos botoes. Compare a complexidade com a versao `StatefulWidget`.



### Pesquisa

Pesquise como `Provider` implementa `InheritedWidget` internamente: por que e preferivel a passar callbacks manualmente por construtores? Como o metodo `select()` pode otimizar reconstrucoes escutando apenas uma propriedade especifica?



### Requisitos

Flutter SDK. Adicionar provider ao `pubspec.yaml`.